

Experiment – 1.3.1

Student Name: Yatharth Bhaskar

UID: 24IBD70014

Branch: AIT-CSE (Data Science)

Section/Group: 24BDS-4 NTPP (A)

Semester: 5th

Date of Performance: 22 July 2026

Subject: Full Stack-II

Subject Code: 24CSP-337

1. Aim:

To design and implement a secure authentication system using JWT for user login and session management.

2. Software Required:

- React.js
- Node.js (optional/mock-based initially)
- JWT libraries (conceptual understanding)
- Browser (for testing)

3. Objective:

- To understand authentication mechanisms in web applications
- To implement token-based authentication using JWT
- To manage user sessions in a stateless architecture
- To handle token storage and validation securely

4. Theory:

Authentication is the process of verifying the identity of a user before granting access to system resources. Traditional session-based authentication relies on server-side session storage, which can become inefficient in distributed systems. JWT-based authentication introduces a **stateless approach**, where the server issues a signed token containing user information. This token is stored on the client and sent with each request for verification.

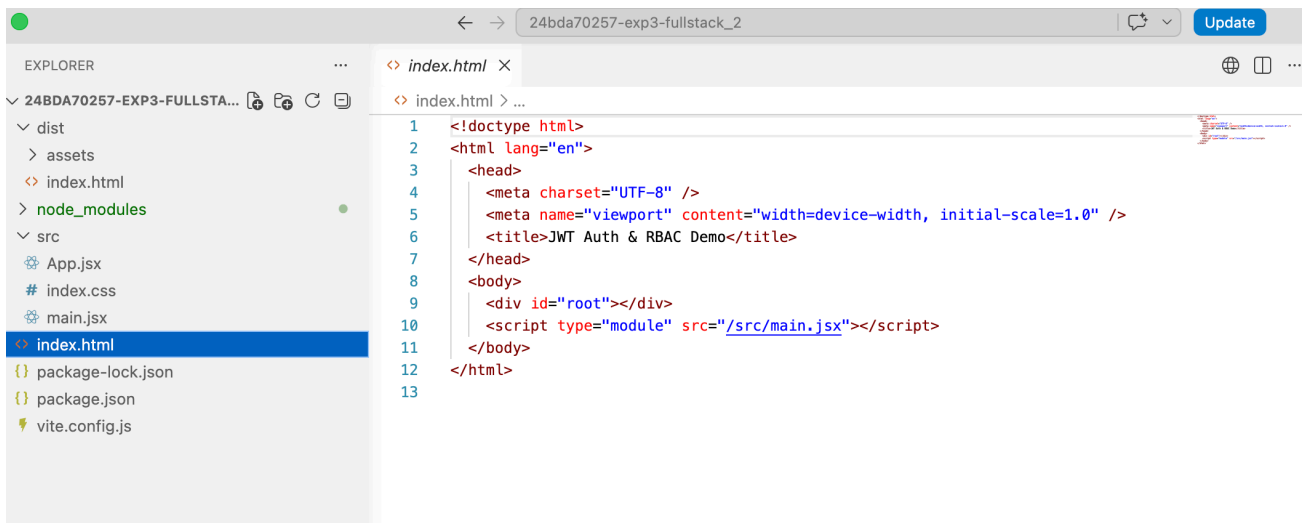
A JWT consists of three parts:

- Header – Defines the algorithm
- Payload – Contains user data (claims)
- Signature – Ensures integrity and authenticity

5. Procedure/Algorithm:

- Create login form UI
- Validate user credentials (*mock or static*)
- Generate or simulate JWT token
- Store token (localStorage/sessionStorage)
- Attach token to requests
- Decode token to extract user information

6. Code:



```
1 <!doctype html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>JWT Auth & RBAC Demo</title>
7   </head>
8   <body>
9     <div id="root"></div>
10    <script type="module" src="/src/main.jsx"></script>
11  </body>
12 </html>
13
```

7. Output:

JWT Auth + RBAC

Login

Secure Login

This demo uses a mock JWT stored in localStorage to simulate stateless authentication.

Username

Password

Login

Invalid credentials. Try admin/admin123, editor/editor123, or viewer/viewer123.

8. Conclusion:

- User login system implemented
- Token-based session handling achieved
- Stateless authentication flow demonstrated

9. Learning Outcomes:

- Understand the fundamentals of user authentication.
- Learn the working principle of JWT-based authentication.
- Identify the three main components of a JWT.
- Implement secure authentication using JWT tokens.
- Compare stateless JWT authentication with session-based authentication.